

SW39000 DMA 扩展实验完整分析报告

实验日期 2026 年 10 月 5 日 作业 8420952 队列 q_share

本报告分析同一个核组内，从核通过 DMA 在主存与本地存储器之间搬运数据的耗时和吞吐，覆盖 8 B 至 128 KiB 消息、七档活跃核数、地址偏移以及核到主存槽的映射。实验旨在量化读写边界、并发饱和和布局敏感性，为通信开销建模提供参数。

1 实验概述与主要结论

主要结果是：128 B 和 256 B 对齐写入具有显著优势；128 B、64 核 put 偏移 4 B 的耗时约为对齐时的 11.79 倍。128 KiB 对齐 put 的本组最高吞吐出现在 4 核，64 核比该测量峰值低约 28%。固定缓冲基地址下，槽步长和核到地址的排列仍会改变吞吐，读取与写入也呈现不同的逐核快慢模式。

这些结果支持在模型中分别处理读写、对齐、消息大小、活跃核集合与地址布局。它们尚不足以确定物理内存 bank、片上路由或仲裁机制；本实验测量的是主存 DMA，不提供 CPE 间 RMA 的小簇通信参数。

项目	本次记录
硬件标识	cpuinfo 的 cpu revision 为 sw39000
时钟记录	MPE 2100 MHz SPE 2250 MHz IMC 825 MHz gclk 1650 MHz
编译器	swgcc/1473 GCC 7.1.0 目标 sw_64sw6a-sunway-linux-gnu
编译与链接	主核 -mhost -O2 -g 从核 -mslave -msimd -O2 -g 链接 -mhybrid -g
提交资源	队列 q_share MPE=1 CG=1 CPE=64 -cache_size 0 独占程度未确认
实验规模	9856 正式案例与 8 项正确性检查 消息上限 131072 B 即 128 KiB

提交时间为北京时间 2026 年 10 月 5 日 20 时 35 分 52 秒。作业编号来自 job.log；主机环境记录为 sw_hpc_78.qd.sw，不能据此定位实际计算节点或共享资源。平台名称使用结果中可核对的 sw39000 标识，不将其他型号的公开峰值作为本机实测能力。

阅读顺序：第 2 至 6 节给出接口、实验矩阵、地址布局和测量方法；第 7 至 14 节逐项解释八幅图；第 15 节汇总结论、实验限制与复现入口。本文的数值均来自本次实验。

2 DMA 接口与内存配置

主核 从核 核组和本地存储器

MPE 是运行控制程序的主核，负责分配主存、构造参数、启动从核与汇总结果。CPE 是执行搬运循环的从核，文档或日志中的 SPE 也是从核标识。CG 是核组，本实验只使用一个 CG 中的 64 个 CPE。LDM 是每个从核附近的软件管理本地存储空间，DMA 用于主存与 LDM 之间的数据传输。

本实验给每个从核分配不同的主存地址槽，避免多个核写入同一目标地址。get 读取主存的 src 缓冲区到本核 LDM；put 将本核 LDM 的测试数据写到主存 dst 缓冲区。读和写使用独立主存数组，未进行双向同时传输。

模式	数据方向	本实验的调用与等待方式
get	主存 → LDM	athread_dma_get 同步调用
put	LDM → 主存	athread_dma_put 同步调用
iget	主存 → LDM	athread_dma_iget 后立即等待 reply=1
iput	LDM → 主存	athread_dma_iput 后立即等待 reply=1

因此四种模式均为串行发起、逐次等待，每个核的软件未完成请求窗口 W=1。非阻塞模式比较的是接口与显式等待的开销，不测请求流水线、计算通信重叠或队列深度。

完成通知的含义

手册对 DMA 读完成的含义是目标 LDM 收到数据；写完成通知表明源 LDM 数据已被取走。因此 put/iput 的单次耗时不能直接解释为数据最终落入 DRAM 的完整可见延迟。主核在 join 后校验最终写入结果，仍不能证明每一轮写入的物理内存提交时刻。

Cache 与 LDM 容量记录

手册描述从核 LDM 总容量为 256 KiB，可按 Cache、私有 LDM 和连续共享 LDM 配置划分。本次请求 - cache_size 0；连续共享 LDM 保持环境设置，实际分配量未查询。请求值、编译告警和运行成功，不能代替硬件实际配置核实。

每核静态数据缓冲为 131072 B、128 B 对齐，另有参数、回复计数器及结果变量。链接器报告静态 LDM 为 132840 B，超过 128 KiB；本次未使用 -b 将栈搬入 LDM。128 KiB 消息已在本次配置中完成，但不能据此保证所有 Cache 或共享 LDM 配置都可运行。

3 完整实验矩阵与执行过程

正式实验分为 **boundary**、**load** 和 **mapping** 三个阶段，均测 **get**、**put**、**iget**、**iput** 四种模式与 **N=1、2、4、8、16、32、64** 七档负载。**N** 表示执行计时循环的活跃从核数；每次均启动 **64** 核，非活跃核参加计时区外的同步与结果回写。

阶段	消息大小数量	偏移 B	步长与排列数量	案例数
boundary 边界	9	0 4 64 124	1 × 1	4×7×9×4=1008
load 负载	30	0 4	1 × 1	4×7×30×2=1680
mapping 映射	8	0 4	4 × 4	4×7×8×2×4×4=7168

三个阶段的全部消息大小

boundary 的 9 种大小为 64、96、124、128、132、192、252、256、260 B。它细扫 128 B 与 256 B 两侧，以及不同地址起点下的写入边界。

load 的 30 种大小均以字节列出：8、16、32、64、96、124、128、132、192、252、256、260、384、512、768、1024、1536、2048、3072、4096、6144、8192、12288、16384、24576、32768、49152、65536、98304、131072。

mapping 的 8 种大小为 64、128、256、4096、32768、65536、98304、131072 B，对应小消息、4 KiB 与 32 至 128 KiB。各阶段大小均未超过 LDM 数据缓冲容量。本文 KiB=1024 B；图中大小按原始字节或 KiB 标注。

固定条件与执行顺序

boundary 与 **load** 使用步长 131200 B、**identity** 排列；**mapping** 使用第 4 节列出的四种步长和四种排列。活跃核 ID 固定为 0 至 **N-1**，改变 **N** 时没有轮换活跃核位置。4 核即 ID 0、1、2、3，不能等同于 2×2 几何小簇。

全部阶段在一次作业、一个常驻主核进程中依次运行：初始化及分配一次，完成 8 项正确性检查，然后按 **plan.csv** 固定顺序执行三个正式阶段，最后 **halt** 并释放内存。正确性检查为四种模式 × 1 核或 64 核，大小 8 B、重复 10 次、偏移 0 B。

每个正式案例先预热 8 次。**S<1024 B** 时计时循环重复 **R=10000** 次，**S≥1024 B** 时 **R=1000** 次。循环重复用于摊薄测量开销，不能作为独立样本计算置信区间。**shuffle** 只打乱核到槽的对应关系，本次运行顺序没有随机化。

4 主存地址槽与核到槽的映射

槽是主存数组中供一个从核访问的逻辑区域。数据缓冲上限 $C=131072\text{ B}$ ，主存槽步长为相邻槽起点的距离，不等于每轮传输字节数 S 。主存 `src` 和 `dst` 数组均按 128 B 对齐，分别分配 $64\times 262144\text{ B}$ ，即各 16 MiB ；两者共 32 MiB ，另有结果数组。

步长计算	步长 B	128KiB 消息后的间隔 B
$C+128$	131200	128
$C+256$	131328	256
$C+4096$	135168	4096
$2C$	262144	131072

四种步长均是 128 B 的倍数，且足以容纳 128 KiB 数据及最大 124 B 偏移，避免槽间重叠。偏移只施加于主存地址，LDM 数据缓冲起点始终保持 128 B 对齐。因而对齐实验改变的是主存端起点。

排列名称	核 ID tid 对应的槽号 slot
identity	$\text{slot}=\text{tid}$
reverse	$\text{slot}=63-\text{tid}$
transpose	$\text{slot}=(\text{tid} \bmod 8)\times 8+\text{floor}(\text{tid}/8)$
shuffle	固定种子 20261005 的 Fisher–Yates 排列 伪随机状态按源码的 32 位 xorshift 更新

逐核主存地址的计算式为 $\text{address}=\text{base}+\text{slot_of_pe}[\text{tid}]\times\text{slot_stride}+\text{offset}$ 。get 使用 `src`，put 使用 `dst`；各案例 CSV 同时保存两端地址、slot 和映射名称，便于核对。两端基地址整个过程固定为 $\text{src}=0\text{x}500001406000$ 、 $\text{dst}=0\text{x}500002408000$ 。

当 $N=64$ 、步长及偏移固定时，四种排列访问相同的 64 个地址，只改变哪个核访问哪个地址。这有助于区分固定核因素与核地址配对因素。当 $N<64$ 时，只取排列中前 N 个核对应的槽，更换排列同时改变访问地址子集。

改变步长同时改变地址间距、页分布、访问范围等条件；不是单独改变已知物理 bank 的实验。虽然按 8×8 展示软件核 ID 或槽号便于识别模式，槽号二维排版不代表内存物理拓扑，虚拟地址也不能直接还原 DRAM channel 或 bank。

5 计时方法与指标计算

一个案例中的计时流程

主核在计时前初始化数据和结果数组并启动从核。各从核获取参数，写模式初始化 LDM 测试模式，全阵列同步；活跃核做 8 次预热后再次同步。随后每个活跃核读取本地计数器，执行 R 次 DMA 发起与完成等待，再读取计数器。循环内没有 barrier。

计数器接口为 `athread_stime_cycle()`。记核 i 的累计周期为 $T_i = \text{end}_i - \text{begin}_i$ ，单次均值为 $t_i = T_i / R$ 。计时包含接口调用、分支、循环、回复计数器处理、等待和返回码检查，不包含主核分配、数据初始化、`spawn/join`、预热、同步、最终内容校验和 CSV 输出。

指标	计算式与解释
逐核 cycles/op	$t_i = T_i / R$
汇总 cycles/op	$t_{\max} = \max(T_i) / R$ 取活跃核最慢均值
聚合带宽 B/cycle	$BW = N \times S \times R / \max(T_i) = N \times S / t_{\max}$
逐核带宽 B/cycle	$BW_i = S \times R / T_i$
扩展比与效率	$\text{Speedup} = BW(N) / BW(1)$ $\text{Efficiency} = \text{Speedup} / N$
偏移耗时比	$t_{\max}(\text{offset}) / t_{\max}(0)$ 大于 1 表示变慢

扩展比要求模式、大小、偏移、步长及排列均匹配；N 变化仍会改变活跃核和地址集合。汇总带宽用最大本地累计周期作为分母，不是各核带宽直接相加，也不是另测的全组共同起止时间。各核比较自身经过的周期，因此不要求不同核计数器的绝对起点同步。

计算示例与时间单位

boundary 阶段的 128 B、64 核、偏移 0 B put， $R=10000$ ，最大累计周期为 4,394,158。因此 $\text{cycles/op}=439.4158$ ，聚合带宽 $=64 \times 128 / 439.4158 = 18.6429 \text{ B/cycle}$ 。这是有效载荷字节数，不含硬件协议开销。

周期计数器频率尚未独立标定，本文保留 cycles 与 B/cycle。若它确实按 `cpuinfo` 中的 SPE 频率 2.25 GHz 递增，则 $\text{ns/op} = \text{cycles/op} \div 2.25$ ，十进制 $\text{GB/s} = BW \times 2.25$ 。`cpuinfo` 的 timer frequency 100 Hz 是另一项系统时钟记录，不能用来换算本计数器。

6 小消息基线 & 数据完整性

单核对齐时的串行完成耗时

下面使用 load 阶段、N=1、偏移 0 B、默认步长及 identity 排列的数据。小消息的平台区可作为本接口循环的固定开销参考，但没有测量并扣除空循环，所以不能称为纯硬件 DMA 延迟。get 与 put 的完成语义也不同，不能把差值直接当作两个方向的物理传播延迟差。

消息 B	get cycles/op	put cycles/op	iget cycles/op	iput cycles/op
8	441.36	240.03	440.75	234.03
16	441.06	240.03	441.00	234.02
32	441.16	240.03	441.05	234.02
64	441.06	240.04	440.85	234.03
128	441.58	240.39	441.22	240.01
256	454.28	252.34	449.27	246.04

8 至 64 B 时，单核 get 与 iget 约为 441 cycles/op，put 约为 240 cycles/op，iput 约为 234 cycles/op。固定开销使小消息 B/cycle 很低；消息增大后摊薄这一开销。写入到 128 B 及 256 B 时还会受对齐与块边界影响，不能用一个线性模型覆盖全部尺寸。

正确性检查与分析复核

源数据模式为 $(tid \times 17 + j \times 13 + 7) \bmod 256$ ，其中 j 为消息内字节下标。每个案例前主核填充 src 并清零 dst；读模式在计时后由从核检查最终 LDM 内容，写模式在 join 后由主核检查最终 dst。计时循环同时累计接口返回错误。每轮重复内容相同，内容校验覆盖最终结果，不逐轮证明每次传输的数据时序。

复核对象	数量或结果
正式案例与正确性案例	9856 + 8 均有对应原始 CSV
正式与检查的活跃核记录	178816 + 260 = 179076
扩展比与效率记录	8448 与相同条件单核基准交叉检查
完成标记及返回错误	RUN_COMPLETE 与 COMPLETE 均记 9856 原始及汇总 errors 均为 0

分析逐项检查了计划与结果数量、大小与参数、核到槽的排列、地址计算、聚合周期等于活跃核最大周期、带宽公式及汇总与原始记录一致性。数据完整性通过表明可进行性能分析；不等于每个性能点已独立重复，也不消除共享资源和测量波动的影响。

7 写入的大小与偏移边界

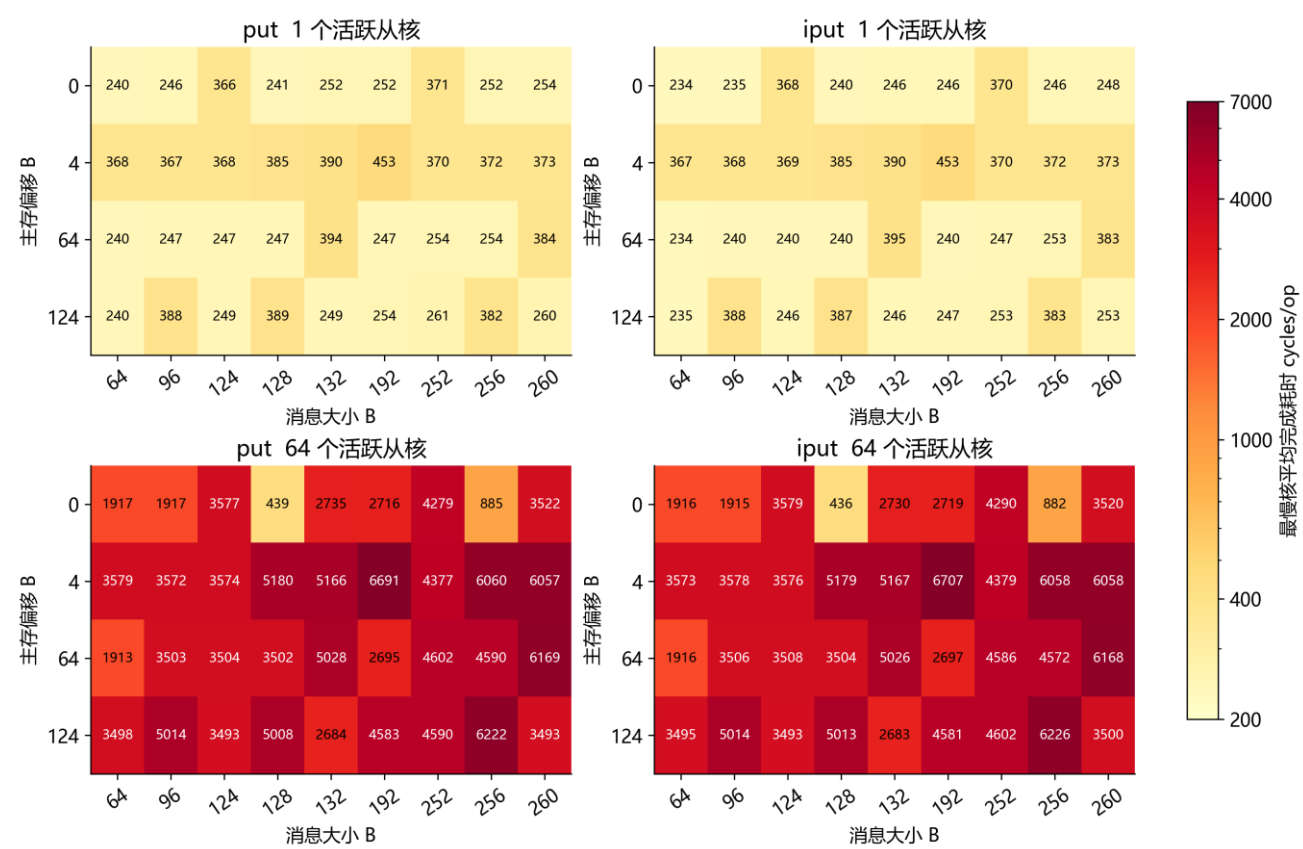


图 1 put 与 iput 的九种尺寸和四种偏移细扫 数值为 cycles/op

64 核 put 消息 B	偏移 0 B	偏移 4 B	偏移 64 B	偏移 124 B
64	1916.82	3578.60	1913.48	3498.34
96	1917.49	3571.86	3503.49	5014.15
124	3577.09	3573.98	3504.05	3492.66
128	439.42	5179.92	3502.36	5008.27
132	2735.08	5166.38	5027.86	2684.06
192	2716.46	6690.86	2694.59	4582.60
252	4279.26	4376.84	4602.30	4589.93
256	885.38	6060.12	4590.04	6222.33
260	3521.90	6057.30	6169.40	3493.38

64 核对齐 put 的 128 B 耗时为 439.42 cycles/op，而相邻 124 B 与 132 B 分别为 3577.09 与 2735.08。256 B 对齐也出现同类低谷，说明转折并非简单的消息越大越慢。

8 偏移代价随负载放大

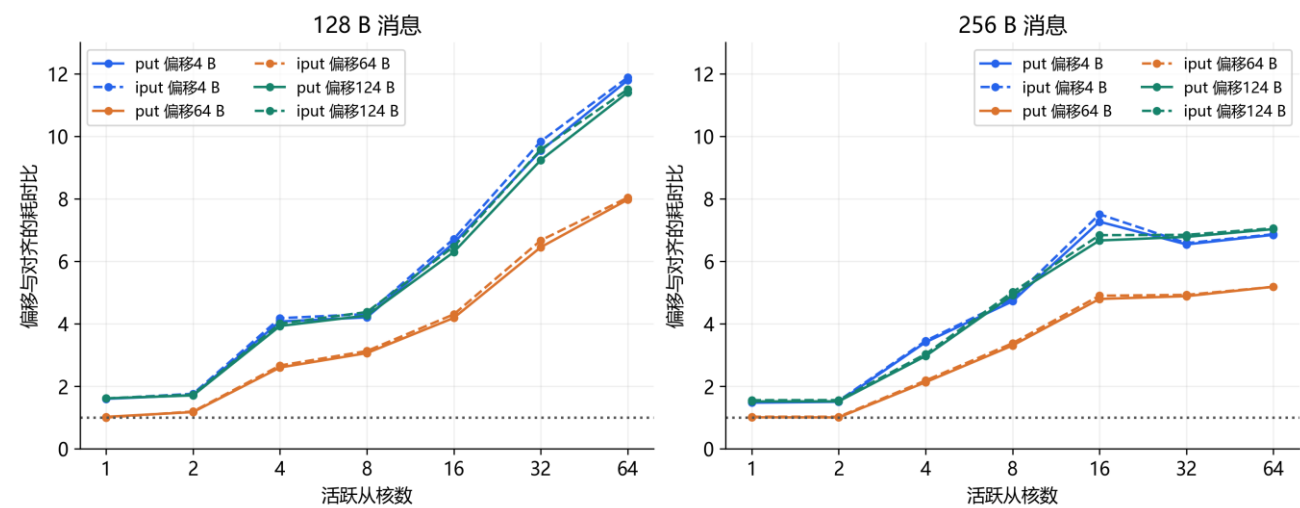


图 2 相同消息下偏移地址相对对齐地址的耗时比 虚线为 i put

模式与核数	消息 B	偏移 4 B 耗时比	偏移 64 B 耗时比	偏移 124 B 耗时比
put 1 核	128	1.599	1.023	1.614
put 64 核	128	11.788	7.970	11.398
put 1 核	256	1.475	1.007	1.516
put 64 核	256	6.845	5.184	7.028
get 64 核	128	1.389	1.390	1.391

128 B 的 put 偏移 4 B 代价从单核约 1.60 倍放大到 64 核约 11.79 倍；后者带宽下降 91.5%。同尺寸 get 的 64 核偏移代价仅约 1.39 倍，读写路径需要分开解释。

仅靠覆盖的 128 B 块数仍不足以解释写入

64 B 偏移 0 B 与 96 B 偏移 4 B 都位于单个 128 B 地址块内，但 64 核 put 耗时分别约为 1916.82 和 3571.86 cycles/op，相差约 1.86 倍。另一方面，64 B 偏移 64 B 耗时约 1913.48 cycles/op，接近偏移 0 B。起点、终点及片段大小也影响写入，不能把额外覆盖一个块直接等价为固定惩罚。

这些现象支持对齐整块与部分块路径、并发争用等候选解释；目前未测硬件事务、bank/channel 或仲裁事件，尚不能确定是读改写、哪级缓存或某个互连部件导致。

9 扩展到 128 KiB 的带宽曲线

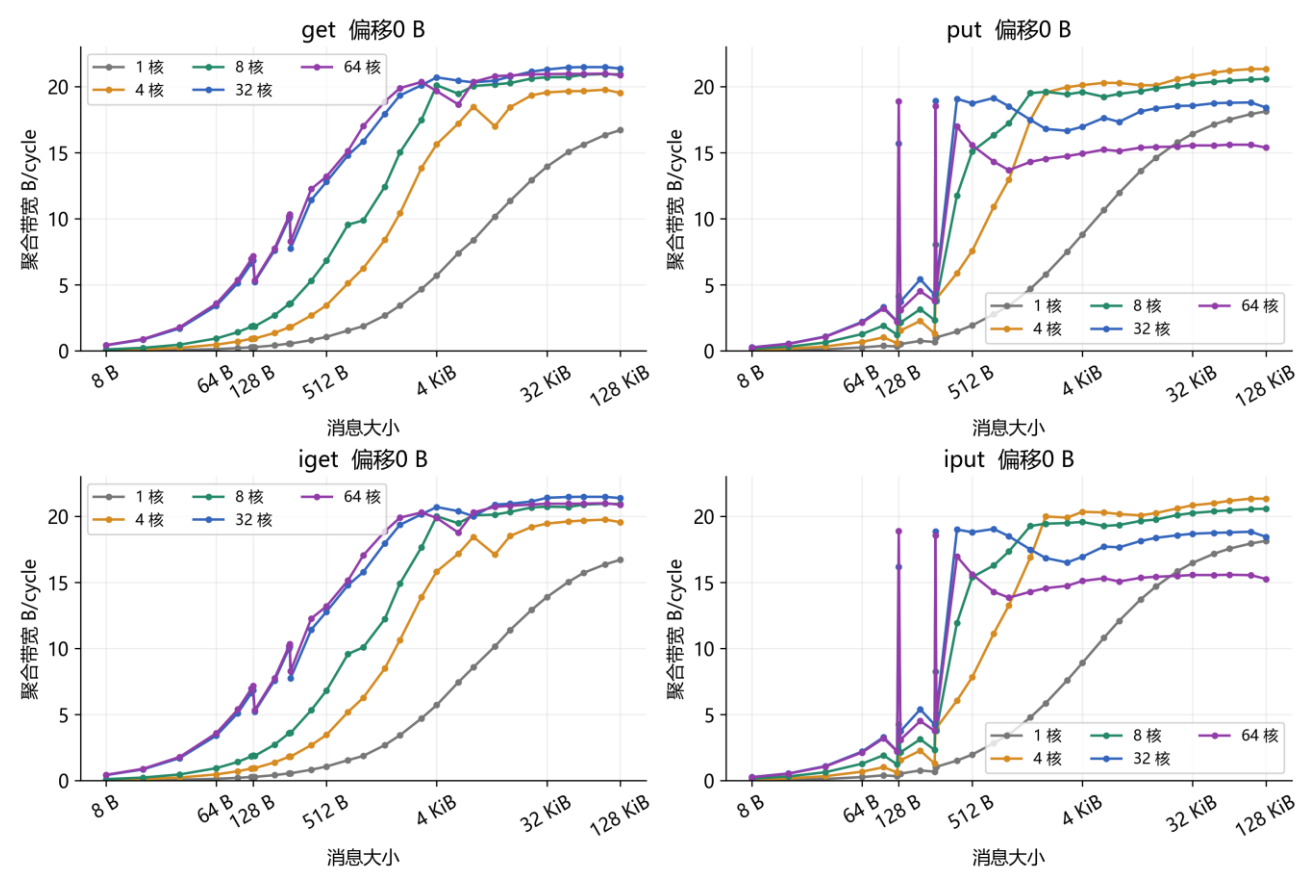


图 3 四种 DMA 模式的对齐传输带宽 展示五档活跃核数

模式	128KiB 单核 B/cycle	128KiB 64 核 B/cycle	64 核与单核 带宽比
get	16.705	20.851	1.248
put	18.120	15.381	0.849
iget	16.704	20.864	1.249
iput	18.137	15.254	0.841

单核 get 从 64 KiB 的 15.614 增到 128 KiB 的 16.705 B/cycle，增幅 7.0%；put 的相应增幅仅 3.5%。固定负载下大消息接近平台区，扩大消息并未带来成倍吞吐收益。

64 核 get 在 32 至 128 KiB 约为 20.85 至 20.98 B/cycle；put 约为 15.38 至 15.60 B/cycle。这是本布局下的有效载荷吞吐，不是全芯片或理论主存峰值。

10 中间核数揭示写入的吞吐下降

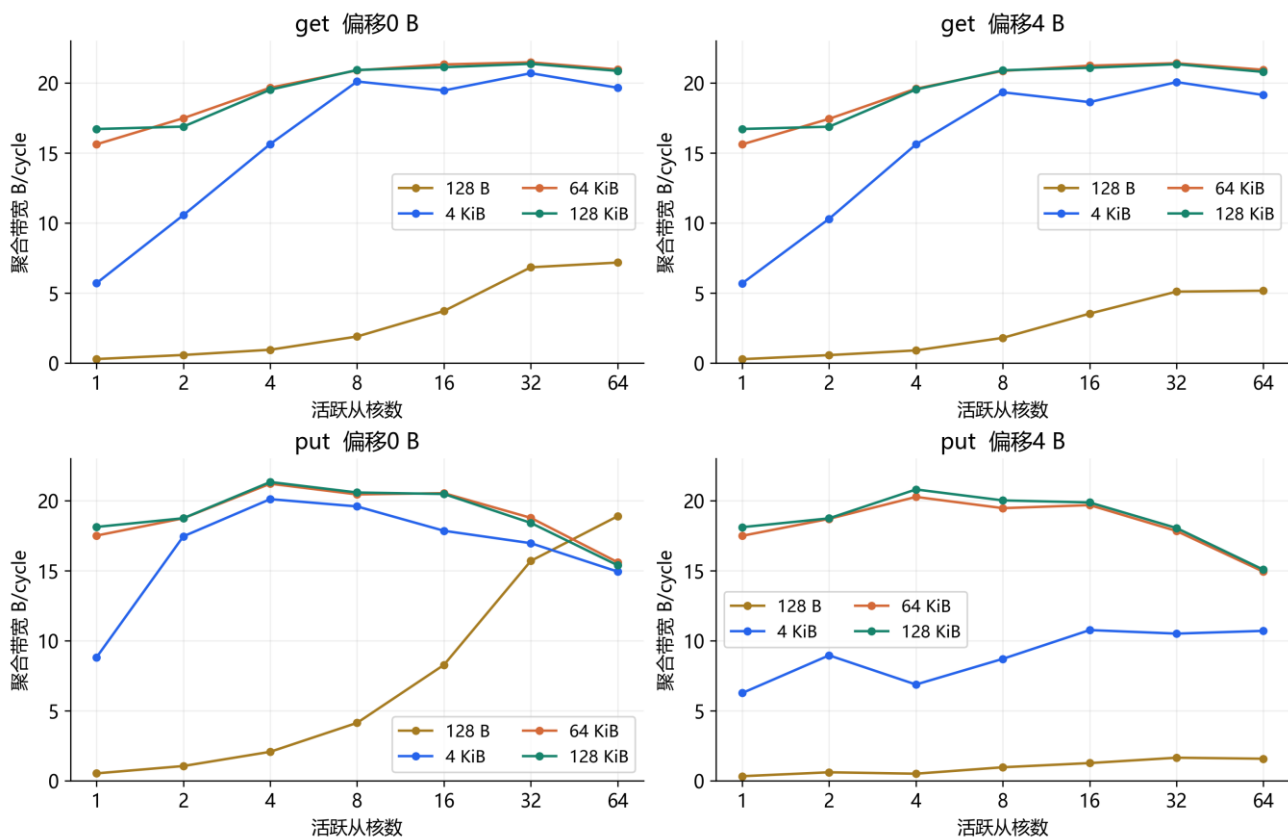


图 4 七档活跃核数的聚合吞吐 消息大小决定饱和和争用表现

活跃核数	get 128KiB B/cycle	put 128KiB B/cycle	put 对 4 核 吞吐比
1	16.705	18.120	0.850
2	16.878	18.741	0.879
4	19.511	21.326	1.000
8	20.918	20.577	0.965
16	21.115	20.471	0.960
32	21.361	18.407	0.863
64	20.851	15.381	0.721

128 KiB 对齐 put 的本组最高值位于 4 核，为 21.326 B/cycle；64 核降至 15.381，比 4 核低 27.9%，也比单核低 15.1%。get 在 8 核已经达到该尺寸本组峰值的 95%，32 核最高，但 8、16、32、64 核之间只有几个百分点差异。

该结论针对固定活跃 ID 与默认槽步长。4 核 ID 为 0、1、2、3，并非几何 2×2 的 0、1、8、9；因此不能将 4 核峰值归因于小簇，也不能用于证明 RMA router 共享结构。

11 槽步长和核到地址排列影响吞吐

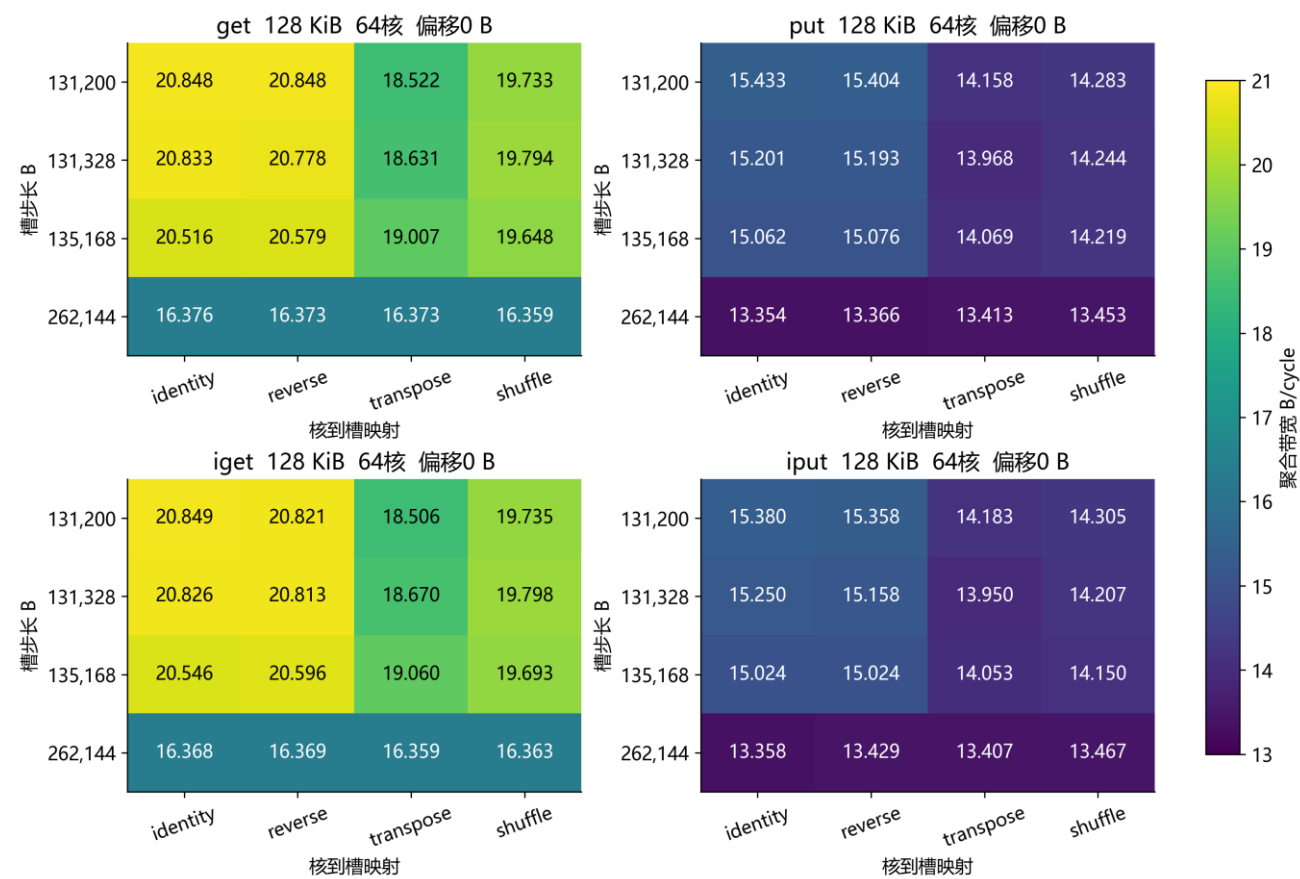


图 5 128 KiB 对齐消息的 64 核槽布局对照 所有排列复用相同缓冲基地址

模式	默认步长 identity	默认步长 transpose	262144 B identity	16 布局最大与最小带宽比
get	20.848	18.522	16.376	1.274
put	15.433	14.158	13.354	1.156

128 KiB、64 核 get 在默认步长的 identity 布局为 20.848 B/cycle，步长改为 262144 B 后为 16.376，下降 21.5%。固定默认步长但改为 transpose 后下降 11.2%；相应 put 下降 8.3%。

两端固定基地址分别为 0x500001406000 与 0x500002408000。64 核时固定步长下更换排列保持地址集合不变，仍出现吞吐差异，说明核到地址的配对关系参与了性能形成。核数小于 64 时排列也改变地址子集，应谨慎区分这两个因素。

改变步长会改变地址间距、页/地址集合和访问范围，不能视为已控制的物理 bank 实验。虚拟地址并不能确定实际 DRAM bank/channel；本数据支持布局敏感性，尚未定位具体映射函数。

12 小消息读取的耗时模式跟随地址槽

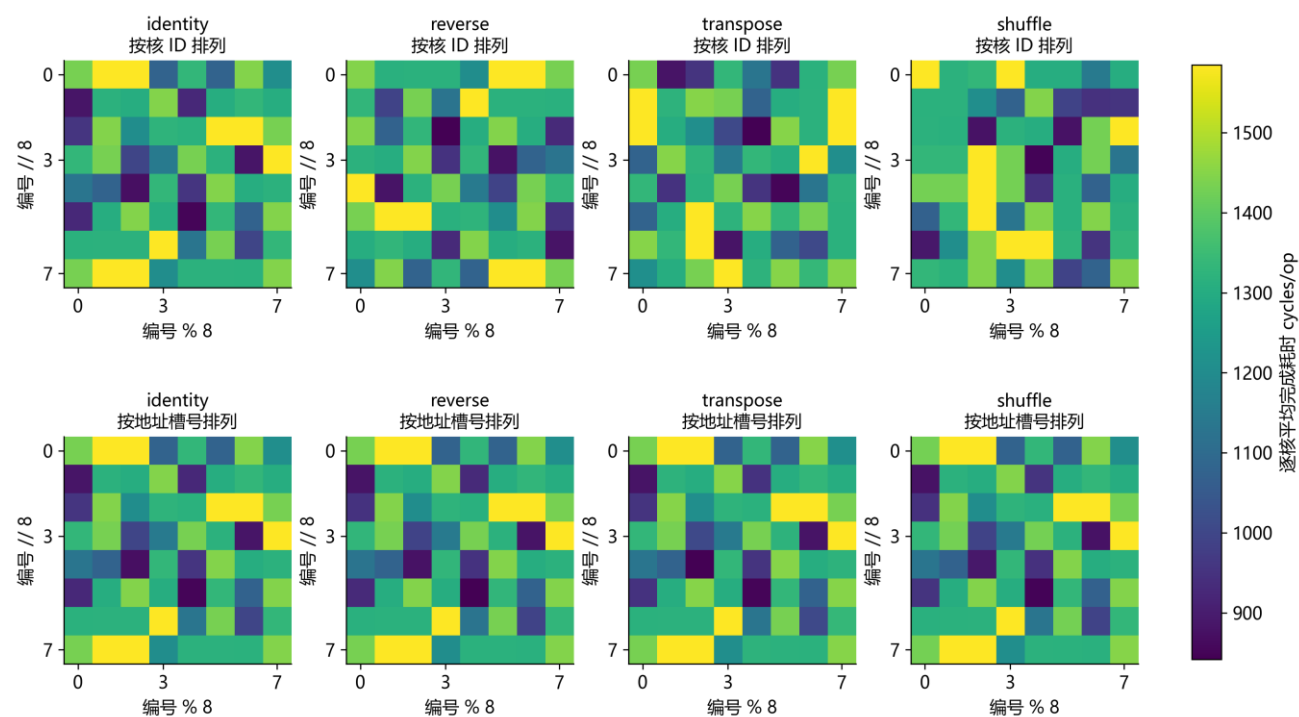


图 6 get 256 B 64 核 偏移 0 B 步长 131200 B 上排按核 下排按槽重排

映射对 identity	按核 ID 的 Pearson r	按地址槽的 Pearson r
reverse	-0.1667	0.9999
transpose	-0.0656	0.9996
shuffle	-0.0895	0.9996

换映射后，按核 ID 观察的快慢模式明显改变；将同一组耗时重新按地址槽号排列后，三种映射均与 identity 高度一致。按槽的相关系数约为 0.9996 至 0.9999，按核约为 -0.17 至 -0.07。这个案例的主要逐核差异更倾向于与主存地址槽关联。

这里同一地址槽对应的是相同虚拟地址及同一进程中的缓冲区，已排除单纯由 tid 编号固定产生全部模式的解释。但尚无法区分内存映射、地址相关路由或其他与地址绑定的资源。大消息 get 的相关性更复杂，不能把本案例推广成所有读取都只由地址决定。

下排的 8×8 仅将槽号按 slot=8r+c 重新排版，不是内存的物理二维拓扑；上排也仅按软件核 ID 展示。Pearson r 是描述性相似度，不是独立重复的显著性检验。

相关系数按两组 64 项耗时的中心化乘积之和，除以各自中心化平方和乘积的平方根计算。r 接近 1 表示快慢变化线性同向，接近 -1 表示反向，接近 0 表示缺少线性一致性；它不要求绝对耗时相等。按槽比较前先把各核耗时重新放到它实际访问的 slot，避免把排列本身当成性能变化。

13 大消息写入保留核位置相关模式

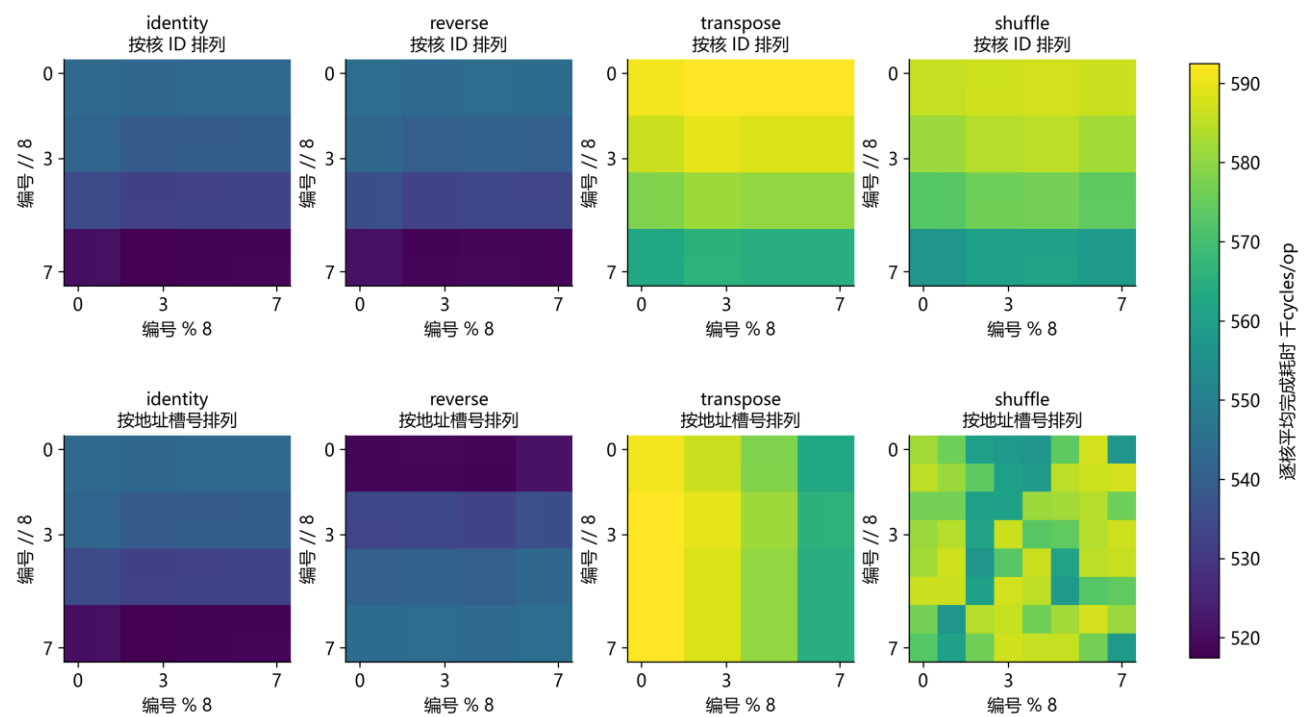


图 7 put 128 KiB 64 核 偏移 0 B 步长 131200 B
上排按核 下排按槽 色条单位为千 cycles/op

映射对 identity	按核 ID 的 Pearson r	按地址槽的 Pearson r
reverse	0.9998	-0.8171
transpose	0.9789	-0.0005
shuffle	0.9760	-0.0890

这个写入案例中，换槽后上排的耗时模式仍相近：按核 ID 的相关系数为 0.9760 至 0.9998；按地址槽重排后，相似性反而减弱。与第 12 节的 256 B 读取相比较，它更支持核位置、注入或调度等与 CPE 绑定因素参与逐核差异。

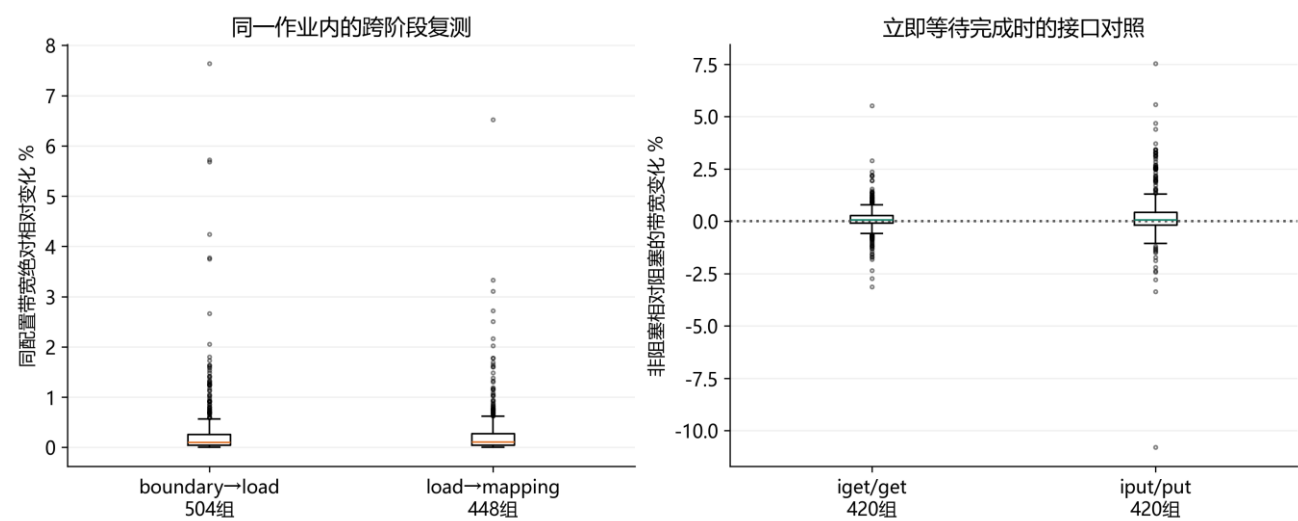
吞吐与模式相似度应分开理解：即使各核的相对快慢排序相近，transpose 与 shuffle 的聚合吞吐仍低于 identity；地址配对仍能改变整体完成速度。固定核因素与地址因素可能同时存在，相关性并未证明具体部件或传输路线。

也不是所有 put 都呈现这一模式：例如 64 B 写入，reverse 与 identity 的按核相关系数约为 -0.81，按槽约为 0.83。写入应按消息规模和对齐条件分别分析，不能仅用一个“核位置惩罚”解释所有数据。

对体系结构建模的直接含义

建议至少区分读与写、整块与部分块、消息大小、活跃核集合、槽步长和映射。现阶段可以使用这些测量条件下的分段拟合或查表；跨布局或跨型号预测仍需验证。本实验没有发起 CPE 间 RMA，不能据此生成小簇通信参数。

14 跨阶段复测和接口对照



匹配阶段	匹配组数	绝对变化中位数	绝对变化 P90	绝对变化最大
boundary→load	504	0.098%	0.735%	7.64%
load→mapping	448	0.108%	0.682%	6.53%

匹配项使用相同模式、大小、核数、偏移、默认步长和 identity 映射。绝对变化定义为 $|BW_{后}/BW_{前}-1|$ 。它们是同一作业内不同阶段的复测，执行顺序固定；虽显示多数配置相近，却不等价于独立作业重复、随机运行顺序或置信区间。几个百分点的细小差异应留待复测。

420 组匹配配置中，iget/get 带宽比中位数为 1.0006，iput/put 为 1.0005。非阻塞接口立即等待时，没有呈现稳定的整体吞吐优势；这不能用于评估多个未完成请求的流水收益。

匹配带宽比	最小值	中位数	P90	最大值
iget/get	0.9686	1.0006	1.0067	1.0551
iput/put	0.8920	1.0005	1.0211	1.0753

图 8 的箱线图汇集不同配置的变化分布，箱体、分位数与范围描述的是这批配置的异质性，不是同一配置多次独立测量的误差条。不能依据比值中位数接近 1，声称接口在所有配置下性能完全相同；也不能从最大单点优势选择出普遍更快的接口。

15 结论 实验限制与复现入口

可用于本次配置的经验参数

单核小消息接口循环的固定耗时约为读 441、写 234 至 240 cycles/op。并发写入存在强烈的大小与偏移交互：128 B、64 核 put 偏移 4 B 后耗时约增至 11.79 倍。128 KiB 对齐 put 的 4 核观测峰值为 21.326 B/cycle，64 核为 15.381；同尺寸 get 在 8 核已接近本组平台区。

默认步长的 128 KiB、64 核 get 约为 20.848 B/cycle，步长改为 262144 B 后下降约 21.5%。256 B 读取的逐核模式在所选映射对照中主要随槽变化，大消息写入主要保留核 ID 相关模式。建模可从这些条件下的查表或分段拟合开始，跨布局与跨型号预测需额外验证。

已经覆盖的条件与尚未覆盖的条件

已经覆盖九种边界尺寸、四种偏移、七档负载、四种槽步长、四种核到槽排列，以及至 128 KiB 的消息大小。但每轮重复使用同一主存地址与同一 LDM 缓冲；更改槽间距不是轮转主存工作集，也没有独立缓冲的 $W>1$ 未完成请求窗口。

实际 Cache 与共享 LDM 分配、计算资源独占程度、同节点其他作业、计数器频率均未完全核实。实验也没有独立作业重复、随机化运行顺序、活跃核位置轮换、硬件事务计数、跨 CG 传输或 RMA 测量。第 14 节的小幅复测差异不能代替这些控制条件。

下一步先用独立作业重复并打乱配置顺序，记录调度分配与共享情况，复测写入边界、4 核和 64 核大消息差异；再固定可比地址集合轮换活跃核集合，测请求窗口和轮转工作集，标定计数器。之后才适合解释具体内存映射、共享部件或通信路径。

原始数据与可复现分析

原始目录为 dma_results_20261005_203552_32484。plan.csv 给出每个案例的条件和文件名；slot_maps.csv 保存实际排列；raw/case_*.csv 保存逐核与 aggregate 行；study_summary.csv 保存 9856 项汇总；study_scaling.csv 保存与单核基准匹配的扩展比及效率。source 保存运行版本源码，build_info.txt、build.log、cpuinfo.txt 与 job.log 提供配置证据。

本次远端实验入口为 `DMA_MAX_BYTES=131072 bash run_dma_boundary.sh q_share`。复核既有结果无需提交作业，可在项目根目录执行以下命令。生成表保存在指定分析目录，图表和本报告的生成脚本保存在本报告目录的 _build 子目录。

```
python bench/analyze_dma_study.py dma_results_20261005_203552_32484 --out
outputs/dma_study_analysis_20261005_203552_32484
```

verification.json 记录数量、校验结果和文件摘要；其余衍生表包括 boundary_comparisons.csv、load_peaks.csv、layout_comparisons.csv、phase_overlap_comparisons.csv、pe_slot_correlations.csv。接口与内存配置依据目录内《SACA 编程指南》DMA 与计时章节，以及《神威众核编程指南》2.4.2、3.4、4.1、4.3 节；实测参数以本次源码和日志为准。